

LearnForge

Text-to-Learn — AI-Powered Course Generator

Complete Project Documentation & Interview Guide

A full-stack, AI-powered web application that turns any topic prompt into a structured, multi-module online course — with lessons, quizzes, videos, Hinglish audio, resources, and PDF export.

React + Vite | Spring Boot (Java 21) | MongoDB | Auth0 | Google Gemini | YouTube Data API

Contents

- 1. Project Overview & Elevator Pitch
- 2. Core Features
- 3. Technology Stack (and why)
- 4. System Architecture
- 5. Data Model (Course → Module → Lesson)
- 6. End-to-End Flows
- 7. Backend Deep Dive (layers & patterns)
- 8. Frontend Deep Dive
- 9. Security
- 10. Deployment & CI/CD
- 11. Testing
- 12. Challenges Faced & Solutions
- 13. Interview Q&A Bank
- 14. Key Numbers & Facts to Remember
- 15. Future Improvements

1. Project Overview & Elevator Pitch

What it is: LearnForge is a full-stack, AI-powered course generator. A user types any topic (e.g. “Intro to React Hooks” or “Basics of Copyright Law”) and the app uses a Large Language Model (Google Gemini) to generate a complete, structured online course.

What it generates for every course:

- A comprehensive course title and description, plus tags.
- 3 to 6 modules, each containing 3 to 5 lessons.
- For each lesson: clear objectives, rich content (headings, paragraphs, code), an embedded YouTube video, 4-5 interactive multiple-choice quizzes with explanations, and suggested reading / resource links.

Extra capabilities: per-user persistence in MongoDB, Hinglish translation with real text-to-speech narration, and one-click PDF export of any lesson. Everything is secured behind Auth0 login.

30-second pitch: “Prompt in → structured AI course out, saved per user, with videos, quizzes, Hinglish audio, and PDF export. Built with React + Spring Boot + MongoDB + Gemini, secured with Auth0, deployed on Vercel + Render + MongoDB Atlas with GitHub Actions CI.”

2. Core Features

Feature	Description
Prompt to Course	Free-form topic generates a full structured curriculum via AI.
Rich lessons	Objectives, text, code, video, and MCQ blocks rendered from structured JSON.
Interactive quizzes	4-5 MCQs per lesson with instant answer checking and explanations.
YouTube videos	Relevant educational videos fetched per lesson via YouTube Data API v3.
Suggested resources	2-4 further-reading / external links per lesson.
Hinglish + TTS	AI translation to Hinglish plus real Gemini text-to-speech audio narration.
PDF export	Download any lesson as a styled, offline PDF (html2canvas + jsPDF).
Secure login	Auth0 (OAuth2 / OIDC) with JWT-protected backend APIs.
Persistent courses	Courses saved per user; reopening loads instantly from the database.

3. Technology Stack (and why)

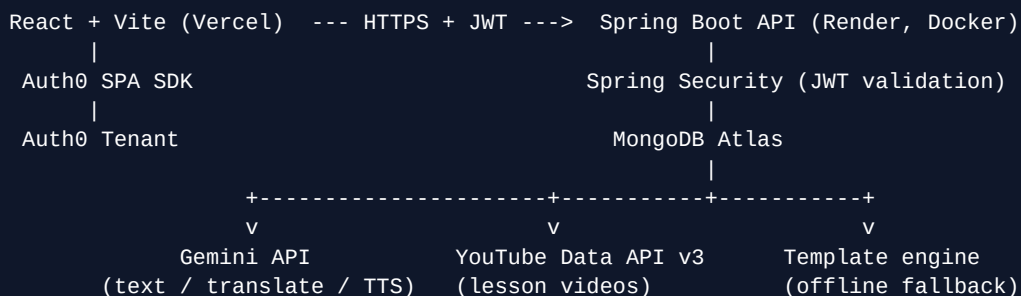
Layer	Technology	Why it was chosen
Frontend	React 19 + Vite	Component model + fastest dev/build tooling (HMR, ESBuild/Rollup).
Routing	React Router 7	Nested routes and shared layout via <Outlet>.
Auth (client)	@auth0/auth0-react	OAuth2 Authorization Code + PKCE, token storage, silent refresh.
PDF	html2canvas + jsPDF	Client-side DOM to image to PDF; no server load.

Layer	Technology	Why it was chosen
FE testing	Vitest + Testing Library	Vite-native, fast unit tests.
Backend	Spring Boot 4.1, Java 21	Strong typing, mature security, DI, large ecosystem.
API security	Spring Security OAuth2 Resource Server	Standard JWT validation against Auth0.
Persistence	Spring Data MongoDB	Document model fits dynamic lesson JSON.
Database	MongoDB (Atlas in prod)	Schema-flexible; lesson content is dynamic blocks.
AI	Gemini (primary), OpenAI (alt), template	Free-tier LLM; pluggable via strategy pattern.
Video	YouTube Data API v3	Real educational videos per lesson.
Deploy	Vercel + Render (Docker) + Atlas	Managed, free-tier friendly, auto-deploy from Git.
CI/CD	GitHub Actions	Lint/test/build on every push and PR.

Why Spring Boot instead of the recommended MERN/Node? The brief recommended MERN but explicitly permitted Spring Boot. It was chosen for type safety, first-class security (Spring Security + OAuth2 resource server), and a clean, testable layered architecture. MongoDB was kept because lesson content is dynamic JSON, which document storage handles naturally.

4. System Architecture

The frontend is a stateless SPA that only renders UI and calls REST APIs. The Spring Boot backend owns all business logic, AI orchestration, and persistence. Every request carries a JWT (no server sessions).



- **Stateless backend** -> scales horizontally; each request self-authenticates with a bearer token.
- **Separation of concerns** -> React renders + calls APIs; Spring Boot handles logic + data + AI.
- **Pluggable AI** -> provider selected at runtime via configuration (strategy pattern).

5. Data Model (Course → Module → Lesson)

Three MongoDB collections, linked by reference (IDs), not embedding:

Entity	Key fields
Course	id, title, description, creator (Auth0 sub), tags[], moduleIds[], createdAt, updatedAt
CourseModule	id, title, courseId, lessonIds[], timestamps
Lesson	id, title, objectives[], content[] (JSON blocks), resources[] {title,url}, enriched (bool), moduleId, timestamps

Content blocks (the lesson content[] array) are polymorphic JSON: heading, paragraph, code (with language), video (search query), mcq (question, options, answer index, explanation).

Why referenced, not embedded? Lessons are generated and updated independently (lazy generation), so referencing by ID lets a single lesson be updated without rewriting the whole course, and avoids MongoDB's 16 MB document limit for large courses.

Why creator = Auth0 sub? The sub claim is the stable unique user ID from Auth0; every query is scoped by it so users only see their own courses.

6. End-to-End Flows

A) Course generation

- User submits a topic in PromptForm.
- Frontend POST /api/ai/generate-course (with JWT).
- PromptBuilderService builds a strict-JSON prompt; the configured provider (Gemini/OpenAI/template) generates it.
- CourseOutlineParserService strips markdown fences, parses JSON, and validates (3-6 modules, 3-5 lessons each).
- Frontend POST /api/courses/save-outline -> persists Course + Modules + Lessons (lessons saved as titles only, empty content).

B) Lesson content: lazy generation + persistence (key highlight)

- On opening a lesson, the frontend checks whether it already has content.
- If empty -> POST /api/courses/{courseId}/lessons/{lessonId}/generate: backend calls Gemini, validates (objectives, valid block types, 4-5 MCQs, 2-4 resources), and saves it (enriched = true).
- If already generated -> served straight from MongoDB: no AI call, instant (like reopening a saved chat).
- Optional background pre-warm (off by default to save quota) builds all lessons right after a course is saved.

Interview line: "Lesson generation is lazy + cached. The first open generates and persists; every reopen is instant and free. This balances quality, cost (AI quota), and UX."

C) Authentication

- Login -> Auth0 redirect using OAuth2 Authorization Code flow with PKCE.
- Auth0 returns tokens; the SDK stores them and silently refreshes.
- Frontend attaches the access token as a Bearer JWT on every API call.
- Backend validates JWT signature (via Auth0 JWKS), issuer, and audience (AudienceValidator), then extracts sub to scope data.

D) YouTube videos

- A lesson video block has a search query.
- VideoBlock calls GET /api/youtube?query=... -> backend calls YouTube Data API v3 -> returns an embeddable video -> rendered as an iframe (results cached to save quota).

E) Hinglish + Text-to-Speech

- POST /api/ai/translate-hinglish -> Gemini translates lesson text to Hinglish.
- POST /api/ai/tts -> Gemini TTS returns raw PCM audio -> WavAudioUtil wraps it into a WAV -> base64 -> HTML audio player (text clamped client-side to stay within limits).

F) PDF export

- LessonPDFExporter renders a hidden, print-styled version of the lesson, captures it with html2canvas, and jsPDF writes it to a downloadable PDF.

7. Backend Deep Dive (layers & patterns)

Package layout: controller -> service -> repository -> model, plus dto, config, security, util, exception.

Design patterns / techniques (name these in interviews):

- **Strategy pattern for AI providers:** AI_PROVIDER config switches gemini / openai / template at runtime behind one common flow. Adding a provider is one branch/class, nothing else changes (Open/Closed principle).
- **Robust LLM-output parsing:** parsers strip code fences, deserialize, and validate schema + semantic rules; invalid AI output becomes a clean 400, never corrupt data.
- **Retry with exponential backoff** (HttpRetryHelper): retries transient 429/502/503/504 with increasing delays.
- **Lazy generation + caching** using an enriched flag (cost + latency optimization).
- **Async background pre-warm** (@Async + dedicated thread-pool executor), opt-in.
- **Typed configuration** via @ConfigurationProperties (AiProperties, Auth0Properties, GeminiProperties, YoutubeProperties).
- **Centralized error handling:** @RestControllerAdvice GlobalExceptionHandler -> consistent ApiErrorResponse JSON with correct status codes.
- **Spring profiles:** default (real Auth0), local (permissive decoder for offline dev), test (hermetic), prod (Render).
- **Ownership checks:** every course/lesson op verifies creator == requester sub -> 403 otherwise (prevents IDOR).

8. Frontend Deep Dive

- **Structure:** pages (Landing, Dashboard, Course, Lesson, NotFound), components (+ blocks renderers), context (App, Toast, Auth), hooks (useAuth, useAsync), utils (api, httpClient, validation, routes, lessonText).
- **Auth gate:** AppLayout shows a splash while Auth0 loads -> the Landing/Login page if logged out -> the app (sidebar + topbar + content) if logged in.
- **Data-driven rendering:** LessonRenderer iterates the content[] JSON and maps each type to a dedicated component (HeadingBlock, ParagraphBlock, CodeBlock, VideoBlock, MCQBlock, UnknownBlock).
- **useAsync hook:** unifies loading / error / reload state for all data fetching.
- **Resilient HTTP client:** timeouts + retries; toasts for feedback; skeleton loaders; error boundaries.
- **Interactive MCQs:** client-side answer checking with explanations.
- **Modern UI:** design-token system, gradient hero + landing page, responsive layout.

9. Security

- OAuth2 / OIDC via Auth0, Authorization Code + PKCE (correct SPA flow, no client secret in the browser).
- Backend is an OAuth2 Resource Server: validates JWT signature (JWKS), issuer, and audience (custom AudienceValidator).
- Stateless sessions (SessionCreationPolicy.STATELESS); CSRF disabled safely because auth is bearer-token based, not cookie based.
- CORS locked to the known frontend origin(s) via configuration.
- Data isolation: queries scoped by sub; ownership assertions return 403.

- Secrets (Mongo URI, API keys) come from environment variables; .env is git-ignored.
-

10. Deployment & CI/CD

- Frontend -> Vercel (static build, VITE_* env vars).
 - Backend -> Render via Docker (render.yaml + Dockerfile), env vars in the dashboard, health check at /api/health.
 - Database -> MongoDB Atlas (cloud).
 - CI -> GitHub Actions (.github/workflows/ci.yml): client job (npm ci -> lint -> test -> build) and server job (JDK 21 -> mvnw verify) on every push and PR.
 - Config strategy: same code, different Spring profiles / env vars per environment (12-factor style).
-

11. Testing

- Backend: JUnit + Spring test - context-load test, controller slice test (MockMvc), and pure unit tests for the AI parsers and the retry helper; a hermetic test profile with a stub JWT decoder (no network).
 - Frontend: Vitest unit tests for utilities (validation, routes, lesson text) and the prompt form.
-

12. Challenges Faced & Solutions

Frame each as problem -> approach -> result.

1. LLMs return unreliable / messy JSON.

Built parser + validator services (strip fences, schema validation, semantic rules). Result: bad AI output fails safe as a 400; data integrity preserved.

2. AI is slow and costs quota.

Lazy generation + persistence with an enriched flag; retry-with-backoff for transient 429s; optional background pre-warm. Result: instant reopens, controlled cost, resilience.

3. Free-tier rate limits (429).

Retry/backoff + graceful error surfacing + a config flag to disable pre-warm. Result: graceful degradation instead of crashing.

4. Local dev vs. real Auth0.

Profile-based security (local uses a permissive decoder, default validates real tokens) and a resilient JWKS decoder (lazy fetch + timeouts) so the app doesn't crash if Auth0 is briefly unreachable.

5. Pluggable AI without rewrites.

Strategy pattern behind AI_PROVIDER (gemini / openai / template).

6. Real TTS, not fake audio.

Call Gemini's TTS model, decode the PCM, and wrap it into a valid WAV container for the browser.

13. Interview Q&A Bank

General / behavioral

Q: Walk me through the project.

A: Use the pitch, then the course-to-lesson generation flow and persistence story.

Q: What was the hardest part?

A: Making unreliable LLM output safe (parsing + validation) and the lazy-generation + caching design.

Q: What would you improve?

A: Persist quiz scores / progress, stream AI responses, cache YouTube in Redis, per-user rate limiting, pagination, e2e tests (Playwright), metrics/observability.

Frontend**Q: How do you render dynamic lesson content?**

A: Data-driven: map each block's type to a dedicated component via LessonRenderer.

Q: How do you handle loading and errors?

A: A useAsync hook + skeletons + toasts + an error boundary; the HTTP client retries transient failures.

Q: How does SPA auth work?

A: Auth0 SDK using Authorization Code + PKCE; the access token is attached as a Bearer JWT; tokens refresh silently.

Q: Why Vite over Create React App?

A: Much faster HMR and builds (ESBuild/Rollup) and modern defaults.

Backend**Q: Explain your layered architecture.**

A: controller / service / repository / model, plus dto, config, security, util, exception.

Q: How do you support multiple AI providers?

A: A strategy pattern selected by the AI_PROVIDER setting (gemini / openai / template).

Q: How do you validate AI responses?

A: Parser services + Bean Validation + semantic rules; non-conforming output fails safe as a 400.

Q: How do you handle failures and rate limits?

A: HttpRetryHelper does exponential backoff on 429/5xx; a global exception handler returns consistent error JSON.

Q: How is a lesson generated only once?

A: An enriched flag plus an ownership-checked generate-and-save endpoint; afterwards it is served from the database.

Database**Q: Why MongoDB?**

A: Lesson content is dynamic JSON blocks; a flexible document model fits the course hierarchy.

Q: Embedded vs referenced documents?

A: Referenced by IDs so lessons update independently and to avoid the 16 MB document limit.

Q: How do you model the hierarchy?

A: Course.moduleIds -> Module.lessonIds -> Lesson, reassembled and ordered in the service layer.

Security**Q: How do you secure the API?**

A: OAuth2 resource server validating JWT signature/issuer/audience; stateless; CORS; per-user data scoping; ownership 403 checks.

Q: Where are secrets stored?

A: Environment variables / git-ignored .env locally; Render and Vercel dashboards in production.

Q: Do you need CSRF protection?

A: No - auth is token-based with no cookies, so CSRF is disabled deliberately.

AI / prompting

Q: How do you design prompts?

A: PromptBuilderService emits a strict JSON schema + rules (module/lesson counts, block types, 4-5 MCQs, resources) and 'return raw JSON only'.

Q: How do you keep output structured?

A: Prompt constraints plus server-side parsing/validation; reject anything non-conforming.

Q: How do you optimize token cost?

A: Generate lessons lazily, cache them in the DB, and clamp text sent to translate/TTS.

System design / scaling

Q: How would this scale to many users?

A: Stateless backend scales horizontally behind a load balancer; index MongoDB on creator; cache YouTube (Redis); queue + background workers for AI; per-user rate limiting; CDN for the SPA.

Q: What is the main bottleneck?

A: LLM latency and quota - mitigated by caching, async generation, and retries.

14. Key Numbers & Facts to Remember

- 3-6 modules per course; 3-5 lessons per module (validated).
- 4-5 MCQs and 2-4 resource links per lesson (validated).
- Content block types: heading, paragraph, code, video, mcq.
- Stack: Java 21, Spring Boot 4.1, React 19, MongoDB, Auth0, Gemini, YouTube Data API v3.
- Endpoints: /api/ai/{generate-course, generate-lesson, translate-hinglish, tts}; /api/courses/{save-outline, my, my/full, {id}, {id}/lessons/{id}/generate}; /api/youtube; /api/health.

15. Future Improvements

- Persist quiz results and per-user learning progress.
- Stream AI responses token-by-token for faster perceived generation.
- Cache YouTube lookups in Redis; add per-user rate limiting.
- Pagination for large course lists; full-text / vector search for related courses.
- End-to-end tests (Playwright) and observability (metrics, tracing, structured logs).
- Editable courses (add/remove modules and lessons) and shareable public course links.